Aegis Entropy: AI-Based Intelligent System for Network Security Monitoring & Automated Response

MODULE Artificial Intelligence	**STATUS** Academic Project
ATTACK FOCUS Port Scanning & Network Reconnaissance	**SEVERITY** MEDIUM-HIGH

Project Team

- El Kartouti Anas
- Hammoubel El Yazid
- Lekhbioui Adil
- Moulay Anas
- Nafia Khadija

Part 1 - Architecture Upgrades: Blueprint vs. Reality

The original needs-expression outlined a standard defensive posture. The final implementation elevates this to a proactive, highly aggressive defense system called **Aegis Entropy**. The table below maps every original design choice to its upgraded counterpart, along with the technical rationale.

Component	Original PDF Specification	Aegis Entropy Upgrade
Response Engine	Linux iptables DROP rules via Python subprocess. Passive approach: the attacker's packets are silently discarded.	**TCP Tarpit.** Forged SYN-ACK with Window Size = 0 is returned, freezing tools and exhausting resources.
MTD Mechanism	Dynamic rotation of service port assignments and virtual IP addresses.	**Active Fingerprint Mutation.** Traffic intercepted inline; TCP/IP headers rewritten to falsify OS identity.
Deception Layer	Full standalone honeypot VMs: Cowrie, Dionaea. High overhead.	**Inline Shadow Nodes** built with Python + Scapy. Fake responses generated directly from the network edge.

Part 2 - Deep Dive into Aegis Entropy

Pillar 1: Core Infrastructure - Asynchronous Interception via NFQUEUE

Mechanism	When a packet reaches the NIC, the Linux kernel pauses its journey and hands it directly to the Python script.
Verdict Power	The packet cannot proceed until the script issues one of three verdicts: ACCEPT, DROP, or MODIFY.

Pillar 2: Sabotage Engine - TCP Tarpitting

When the AI model flags an active Nmap scan, Scapy intercepts it and crafts a forged SYN-ACK with Window Size = 0. The attacker's machine interprets this as: 'The server is alive but its buffer is full, I must wait.' Nmap will literally hang.

Pillar 3: Deception Engine - Active Fingerprint Mutation

The `network_mutator.py` script intercepts outgoing packets and dynamically rewrites fields (TTL, TCP Options) before they reach the attacker, creating conflicting OS signatures from a single VM.

1. Executive Summary & 2. Problem Statement

This document presents the formal decision-making needs expression for the design, implementation, and validation of an AI model dedicated to the real-time detection and automated mitigation of Port Scanning and Network Reconnaissance activity.

Beyond detection, this project integrates Moving Target Defense (MTD) and Honeypots. Together, detection, deception, and surface mutation form a layered, intelligent security posture to counter adversaries who probe networks to map active hosts and enumerate open ports.

3. Project Objectives & 4. Functional Requirements

Primary Objective	Train and deploy an AI model capable of detecting port scanning in real time.
Scan Type Coverage	Detect SYN, TCP Connect, UDP, ACK, XMAS, and slow/distributed scans.
Automated Response	Enforce automated TCP Tarpitting and Active Fingerprint Mutation.

5. Dataset Specification

Dataset	Relevant Subset	Approx. Samples	Source
CICIDS2017	PortScan category	~158,000 flows	UNB/CIC
UNSW-NB15	Reconnaissance category	~13,000 records	UNSW Australia
CAIDA	Backscatter/SYN flood probes	Variable	CAIDA.org

6. Key Network Features for the ML Model

Feature Name	Description	Relevance
Distinct Dst Ports/IP	# unique destination ports contacted by source	Critical-core scanning signature
SYN Flag Count	SYN packets sent without completing handshake	Critical stealth SYN scan
RST Flag Count	RST responses from closed ports	Very High-port probe rejection
Flow Duration	Duration of each individual probe connection	High-very short in fast scans
Total Fwd Packets	Packets per flow from scanner	usually 1 per probe
ACK Flag Count	ACK packets without prior SYN	Medium-ACK scan detection
IAT Mean	Mean inter-arrival time between probes	High slow scans have large IAT

Feature Name	Description	Relevance
Unique Dst IPs/Src	# distinct destination hosts contacted	Medium host discovery phase
TTL Value	Time-to-live field in IP header	Low-OS fingerprinting attempts
Bwd Packet Length	Response packet size	Medium-RST vs SYN-ACK ratio
Shadow Node Interaction	Binary: did source IP contact a decoy node	high-confidence threat signal Critical
MTD Port Delta	Difference between probed port and current port	reveals stale recon data High
TCP Window Size (outbound)	Window size in attacker-received SYN-ACK	New Tarpit effectiveness metric

7. Machine Learning Model Strategy

Primary Model	Random Forest Classifier (handles high-dimensional scan features, robust to class imbalance).
Secondary Model	XGBoost (high-performance ensemble benchmark).
Slow Scan Detection	Isolation Forest (unsupervised) on sliding 60-second window aggregates per source IP.
Imbalance Handling	SMOTE oversampling on the PORT_SCAN minority class.
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, AUC-ROC, Shadow Node True Positive Rate.

8. MTD & Honeypot Component Specifications (Updated)

8.1 Moving Target Defense - Active Fingerprint Mutation

Mutation Scope	Dynamic rewriting of TTL values and TCP Options (MSS, WScale, SACK, Timestamps order) per destination port to mimic different OS profiles.
Rotation Trigger	Time-based (scheduled interval) AND event-based (confirmed scan detection).
AI Integration	Mutation state table is fed as a live input to the AI model, ensuring traffic directed at fingerprinted ports is correctly classified.

8.2 Honeypot Integration - Inline Shadow Nodes

Implementation	Python socket listeners + Scapy packet crafting (no full OS simulation).
Services Emulated	SSH (port 22), HTTP (port 80), FTP (port 21), custom service banners.
AI Contribution	Shadow Node events are ingested as real-time features, significantly boosting detection confidence.

9. Non-Functional Requirements & Success Criteria

Requirement / Criterion	Target Value / Outcome
Fast Scan Detection	< 8 seconds for aggressive Nmap scan
Slow Scan Detection	< 90 seconds (sliding window)
Model F1-Score & Accuracy	F1 ≥ 88%, Accuracy ≥ 90% (CICIDS2017)
False Positive Rate	< 6%

Requirement / Criterion	Target Value / Outcome
Blocking Response Time	< 15 seconds (TCP Tarpit activation)
MTD Mutation Latency	< 30 seconds (Detection to completed rewrite)
Shadow Node Detection	≥ 60% of scanning IPs contact a node

10. Technical Stack Summary

- **OS / Virtualisation:** Ubuntu Server (IDS + victim nodes), Kali Linux (attacker), GNS3/VirtualBox
- **Packet Interception:** Netfilter Queue (NFQUEUE), Scapy with BPF filters
- **ML Libraries:** Scikit-learn, XGBoost, imbalanced-learn, Pandas, NumPy
- **Web Dashboard:** Flask + Socket.IO + D3.js
- **Active Defense:** TCP Tarpit via Scapy (Window Size = 0), network_mutator.py, Inline Shadow Nodes